Designed by: Dr. Falguni Parsana

MongoDB CRUD Operations – Practical Lab Exercises (Assignment 2)

Subject: Advanced Database Management System | MCA/MSc.IT Students

Practical Title: Implementation of CRUD Operations using MongoDB Shell (mongosh)

Objective: To understand and implement Create, Read, Update, and Delete operations in MongoDB using practical examples.

Practical 1: Create Database and Collection

Task 1: Create a database named **CollegeDB**.

Task 2: Create a collection named **students**.

Task 3: Display the available collections.

```
use CollegeDB
db.createCollection("students")
show collections
show dbs
```

Practical 2: CREATE Operations

Task 4: Insert one student record using insertOne().

```
db.students.insertOne({
        rollno: 101,
        name: "Rahul",
        age: 22,
        course: "MCA",
        semester: 1,
        city: "Rajkot",
        marks: 85
        })
```

Task 5: Insert the following five student records using insertMany().

Roll No.	Name	Age	Course	Semester	City	Marks
102	Priya	23	MCA	1	Ahmedabad	90
103	Amit	22	MCA	3	Rajkot	75
104	Neha	24	MCA	3	Surat	88

105	Karan	21	MCA	1	Jamnagar	65
106	Riya	23	MCA	3	Rajkot	92

Practical 3: READ Operations

Write MongoDB queries to perform the following operations.

Exercise

Display all student records.

Display the details of the student whose name is "Rahul".

Display all students belonging to Rajkot.

Display students having marks greater than 80.

Display students having marks less than 70.

Display students whose age is greater than or equal to 23.

Display students studying in semester 3.

Display only the name and marks of all students.

Display students whose marks are between 70 and 90.

```
db.students.find({marks: {$gte: 70, $lte: 90}})
```

Display students belonging to Rajkot OR Surat.

Display students whose semester is 3 AND marks are greater than 80.

Display students in descending order of marks.

Display the top three students based on marks.

Count the total number of students.

```
db.students.countDocuments({})
```

Display students whose names start with "R".

Method 1: Using Regular Expression (Short Syntax)

```
db.students.find({name: /^R/})
```

Method 2: Using $regex

```
db.students.find({name: {$regex: "^R"}})
```

Practical 4: UPDATE Operations

No.	Exercise	MongoDB Method
1	Update Rahul's marks from 85 to 95.	updateOne()
2	Change Priya's city to Mumbai.	updateOne()
3	Increase marks of all students by 5.	updateMany() with $inc
4	Add a new field status: "Active" to all students.	updateMany()
5	Change the semester of all semester 1 students to semester 2.	updateMany()
6	Rename the field marks to totalMarks for all students.	$rename
7	Remove the age field from Rahul's document.	$unset
8	Replace Karan's entire document with new details.	replaceOne()
9	Update Riya's city to Vadodara and semester to 4.	updateOne()
10	Increase the age of all students by 1.	$inc

Sample solutions

```
// Task 1
db.students.updateOne(
  {name:"Rahul"},
  {$set:{marks:95}}
)
// Task 2
db.students.updateOne(
  {name:"Priya"},
  {$set:{city:"Mumbai"}}
)
// Task 3
db.students.updateMany(
  {},
```

```
 {$inc:{marks:5}}
)

// Task 4
db.students.updateMany(
 {},
 {$set:{status:"Active"}}
)
// Task 5
db.students.updateMany(
 {semester:1},
 {$set:{semester:2}}
)

// Task 6
db.students.updateMany(
 {},
 {$rename:{marks:"totalMarks"}}
)

// Task 7
db.students.updateOne(
 {name:"Rahul"},
 {$unset:{age:""}}
)

// Task 8
db.students.replaceOne(
 {name:"Karan"},
 {
   rollno:105,
```

```
  name:"Karan",
  age:22,
  course:"MCA",
  semester:2,
  city:"Surat",
  totalMarks:80
 }
)

// Task 9
db.students.updateOne(
 {name:"Riya"},
 {$set:{city:"Vadodara",semester:4}}
)

// Task 10
db.students.updateMany(
 {},
 {$inc:{age:1}}
)
```

Practical 5: DELETE Operations

No.	Exercise
1	Delete the student record whose name is "Amit".
2	Delete one student belonging to Surat.
3	Delete all students whose semester is 4.
4	Delete all students having totalMarks less than 75.
5	Delete all students belonging to Rajkot.
6	Delete all records from the students collection without deleting the collection.
7	Delete the students collection completely.

Sample solutions

```
// Task 1
db.students.deleteOne({name:"Amit"})

// Task 2
db.students.deleteOne({city:"Surat"})

// Task 3
db.students.deleteMany({semester:4})

// Task 4
db.students.deleteMany({totalMarks:{$lt:75}})

// Task 5
db.students.deleteMany({city:"Rajkot"})

// Task 6
db.students.deleteMany({})

// Task 7
db.students.drop()
```

Practical 6: Additional Exercises for Students (Without Solutions)

These exercises can be assigned for independent lab practice.

Level 1 – Basic

1. Create a database named LibraryDB.
2. Create a collection named books.
3. Insert 10 book records containing book_id, title, author, price, category, and quantity.
4. Display all books.
5. Find books whose price is greater than 500.
6. Display books belonging to the "Programming" category.
7. Display only the book title and author.

8. Count the total number of books.

Level 2 – Intermediate

9. Display books having a price between 300 and 700.
10. Find books whose quantity is less than 5.
11. Display books in ascending order of price.
12. Display the three most expensive books.
13. Increase the price of all Programming books by 100.
14. Update the quantity of a particular book.
15. Add a new field availability: "Yes" to all books.
16. Rename the field price to bookPrice.
17. Remove the category field from one book.
18. Replace an entire book document using replaceOne().

Level 3 – Advanced

19. Find books whose category is "Programming" and bookPrice is greater than 500.
20. Find books belonging to either "Database" or "Networking".
21. Find books whose titles begin with "M".
22. Increase the quantity of all books by 10.
23. Update a book using upsert: true when it does not exist.
24. Delete one book whose quantity is zero.
25. Delete all books having bookPrice less than 200.

Instructions for Students: Execute all queries in MongoDB Shell (mongosh), verify the output after each operation, and include the query and corresponding output screenshot in the practical journal.